pfi

Python for Fixed Income

COURSE 1 · PYTHON FOUNDATIONS

Introduction to Python, Part 1

WEEK 1, DAY 1, PART 1

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

The desk lead's second note

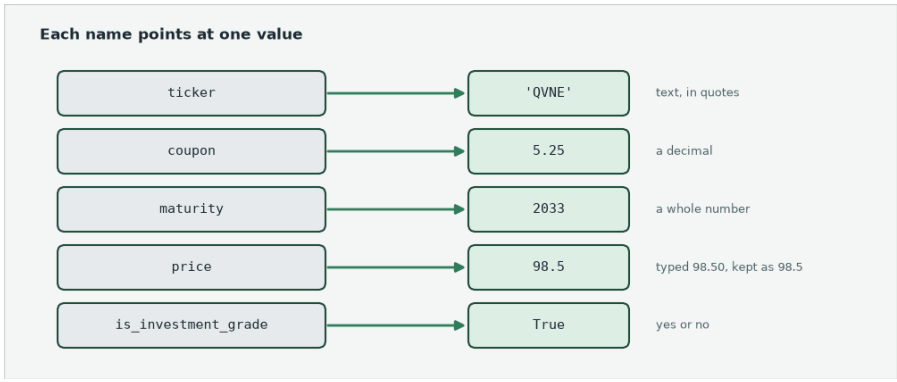
Put our book into Python. What does our first 1,000,000 face of QVNE pay a year, and what did it cost? What is our total face, and our price range? And clean up the broker's messy text.

Ticker	Issuer	Coupon (%)	Maturity	Price	Rating	Face held
QVNE	Quorvane Energy	5.25	2033	98.50	BBB	1,500,000
DNMR	Dunmarrow Rail	6.125	2034	103.25	BB+	500,000
PLWK	Pellwick Paper	4.75	2031	95.00	A-	750,000
HLVF	Halvering Foods	7.50	2032	101.00	B+	250,000

Start with one bond: QVNE.

1.1 Store one bond

```
ticker = 'QVNE'
coupon = 5.25
maturity = 2033
price = 98.50
is_investment_grade = True
print(ticker, coupon, maturity, price, is_investment_grade)
QVNE 5.25 2033 98.5 True
```



Naming rules

Rule	Allowed	Not allowed
Letters, digits, underscores	`is_investment_grade`	`investment grade`
No digit first	`ticker1`	`1st_ticker`
Case counts	`price, Price: two names`	
No reserved word	`rating_class`	`class, if, True`

Excel side by side

Excel's defined names also refuse a leading digit or a space, but Excel treats Price and PRICE as one name.

Four data types, and text that looks like a number

Type	Short for	Holds	Known by	Excel
str	string	text, 'QVNE'	quotes	text
float	floating point	a decimal, 98.5	a decimal point	number
int	integer	a whole number, 2033	no decimal point	number
bool	boolean	True, False	capital, no quotes	TRUE, FALSE

Text plus a number: Python will not guess

```
price_text = '98.50'
price_text + 0.5
TypeError: can only concatenate str (not "float") to str
```

Convert it on purpose

```
print(float(price_text) + 0.5)
print('QVNE at ' + str(price))
99.0
QVNE at 98.5
```

Quotes decide the type.

1.2 A year's coupon, the desk's way

```
face = 1000000
annual_income = face * coupon / 100
print(f'QVNE pays ${annual_income} a year')
print(f'QVNE pays ${annual_income:,.2f} a year')
QVNE pays $52500.0 a year
QVNE pays $52,500.00 a year
```

Piece	Does
`annual_income`	the value to show
`:`	starts the format code
`,`	a comma every three digits
`.2`	exactly 2 decimals
`f`	fixed point

Units matter: the Mars Climate Orbiter

NASA lost it on September 23, 1999. The board's root cause: "the failure to use metric units in the coding of a ground software file". It arrived about 170 kilometers lower than planned.

Source: Mars Climate Orbiter Mishap Investigation Board, Phase I Report, November 10, 1999, pages 6 and 7.

Format codes, beside Excel's number formats

The code

```
print(f'Price: {price:.2f}')
print(f'Coupon: {coupon:.3f} percent')
print(f'Face: {face:,}')
print(f'{"Ticker":<8}{"Price":>10}')
print(f'{ticker:<8}{price:>10.2f}')
```

What it shows

```
Price: 98.50
Coupon: 5.250 percent
Face: 1,000,000
Ticker       Price
QVNE         98.50
```

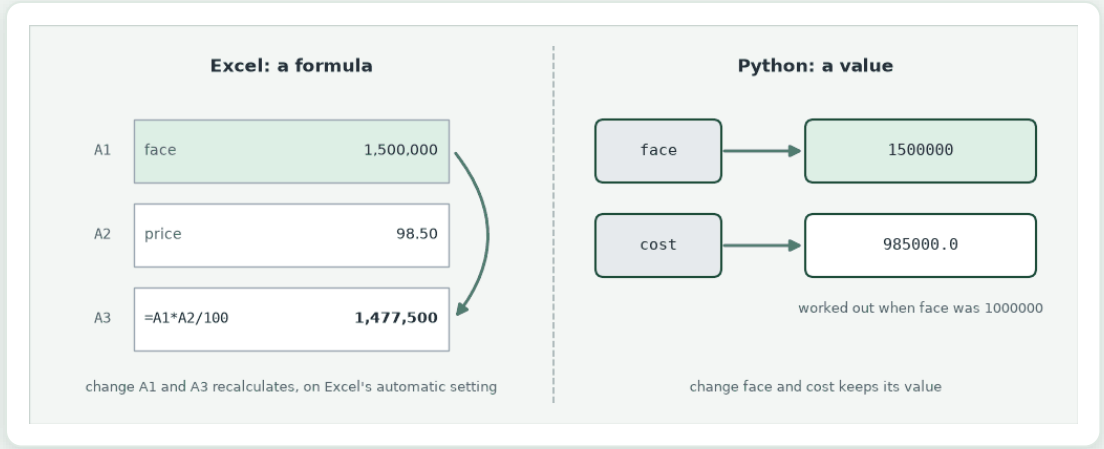
A format code is a number format, like #,##0.00: the look changes, not the value.

Python	Excel	Shows
`f'{annual_income:,.2f}'`	`=TEXT(A1,"#,##0.00")`	52,500.00
`f'{price:.2f}'`	`=TEXT(A1,"0.00")`	98.50
`f'{0.0525:.2%}'`	`=TEXT(A1,"0.00%")`	5.25%
`f'{ticker} pays {coupon:.2f}'`	`=A1&" pays "&TEXT(B1,"0.00")`	QVNE pays 5.25

1.3 Payment, cost, and a new mark

```
payment = annual_income / 2
cost = face * price / 100
print(f'Each payment: ${payment:,.2f}')
print(f'Cost: ${cost:,.2f}')
Each payment: $26,250.00
Cost: $985,000.00
new_price = 99.25
new_value = face * new_price / 100
change = new_value - cost
print(f'Change in value: ${change:,.2f}')
Change in value: $7,500.00
```

Excel side by side. With face in A1 and price in A2, =A1*A2/100; face * price / 100 also says what is multiplied.



cost still holds 985,000.00: a value, not a formula, and Python never recalculates by itself.

The note's second answer: 985,000.00 dollars.

Round lots, a discount factor, and rounding

Round lots

```
order_face = 2350000
lot_size = 1000000
print(order_face // lot_size)
print(order_face % lot_size)
2
350000
```

A discount factor

```
discount_factor = 1 / (1 + 0.05 / 2) ** 10
Discount factor: 0.781198
100 in 5 years is worth 78.12 today
```

Rounding

```
print(round(99.125, 2))
print(round(0.5), round(1.5), round(2.5))
99.12
0 2 2
```

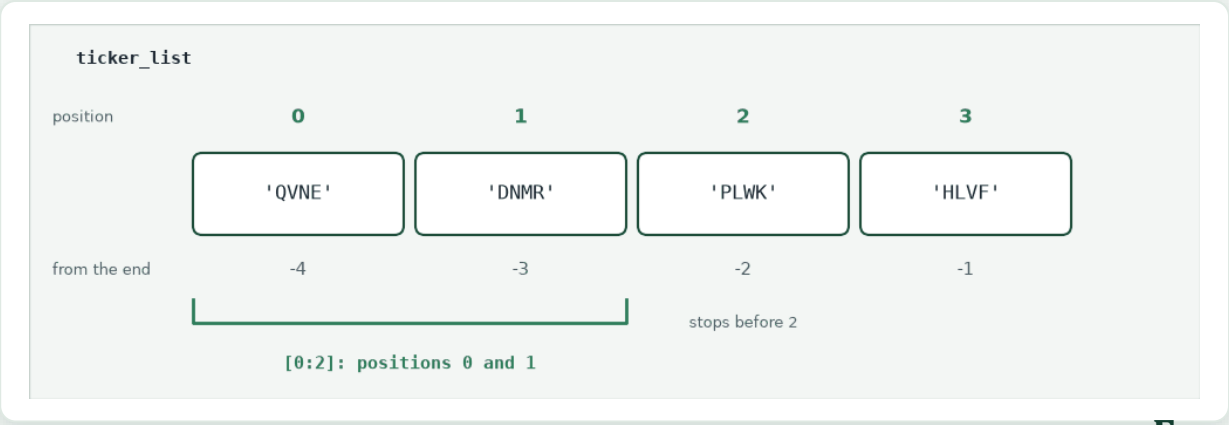
Price	Python round()	Excel =ROUND()
99.125	99.12	99.13
99.375	99.38	99.38
99.625	99.62	99.63
99.875	99.88	99.88
Total	398.00	398.02

Banker's rounding is older than computers

Newcomb's tables of 1883: "write the nearest EVEN number." Python 3.0 adopted it in 2008.

Sources: Newcomb, pp. 14-15; Woodward, p. 42; European Commission; Regulation 1103/97, Art. 5; van Rossum; Peters; What's New In Python 3.0; Wikipedia, Rounding.

1.4 One list per column



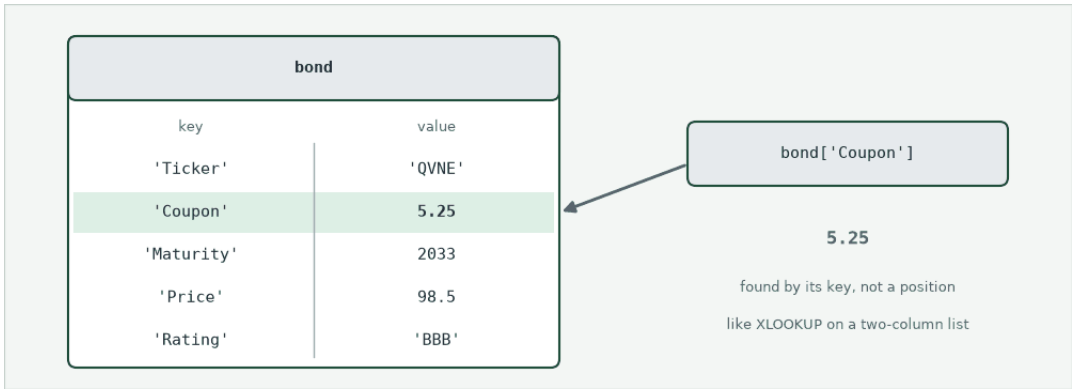

```
print(coupon_list[2])
print(ticker_list[0:2])
print(rating_list[-1])
4.75
['QVNE', 'DNMR']
B+
```

```
total_face = sum(face_list)
average_coupon = sum(coupon_list) / len(coupon_list)
Bonds: 4
Prices from 95.00 to 103.25
Total face: 3,000,000
Average coupon: 5.906 percent
```

Excel side by side

Python	Excel
`len(price_list)`	=COUNTA(A1:A4)
`min(price_list), max(price_list)`	=MIN(A1:A4), =MAX(A1:A4)
`sum(face_list)`	=SUM(A1:A4)
`sum(coupon_list) / len(coupon_list)`	=AVERAGE(A1:A4)

A dictionary: one bond by name




```
bond = {
    'Ticker': 'QVNE',
    'Coupon': 5.25,
    'Maturity': 2033,
    'Price': 98.5,
    'Rating': 'BBB',
}
print(bond['Coupon'])
5.25
```

```
bond['Seniority']
KeyError: 'Seniority'
print(bond.get('Seniority'))
print(bond.get('Seniority', 'not recorded'))
print(bond.get('Coupon'))
None
not recorded
5.25
```

Excel side by side

Keys in A1:A6, values in B1:B6:

```
=XLOOKUP("Seniority",A1:A6,B1:B6,"not recorded")
```

returns the fallback, as get() does.

The whole book, in one name, then drawn

`portfolio['Price'][1]`

	Ticker	Price	Face
0	QVNE	98.5	1500000
1	DNMR	103.25	500000
2	PLWK	95.0	750000
3	HLVF	101.0	250000

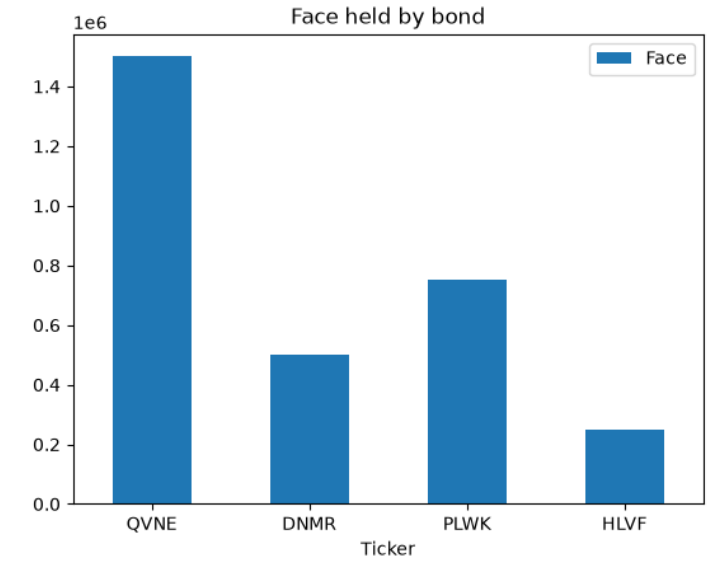
1. the column, by its key
2. the bond, by its position

`holdings[1]['Price']`

	Ticker	Price	Face
0	QVNE	98.5	1500000
1	DNMR	103.25	500000
2	PLWK	95.0	750000
3	HLVF	101.0	250000

1. the bond, by its position
2. the column, by its key

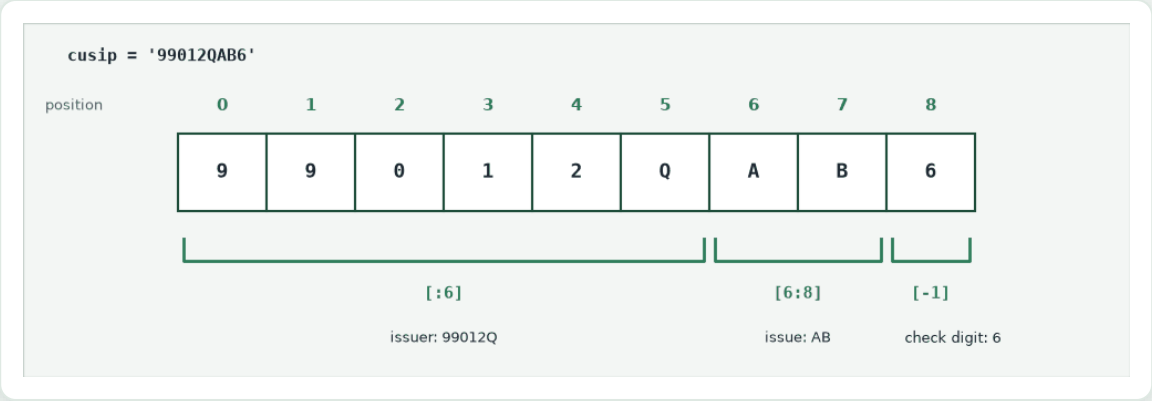
Brackets	Container	Picks by	Can change
[ ]	list	position	yes
( )	tuple	position	no
{ }	dictionary	key	yes



QVNE is half the book: 1,500,000 of the 3,000,000 face.

The 1e6 above the axis means times 1,000,000, as e+04 did in 1.2: the bar at 1.5 is 1,500,000.

1.5 Take a CUSIP apart, clean the broker's text




```
cusip = '99012QAB6'
print(len(cusip))
print(cusip[:6])
print(cusip[6:8])
print(cusip[-1])
9
99012Q
AB
6
```

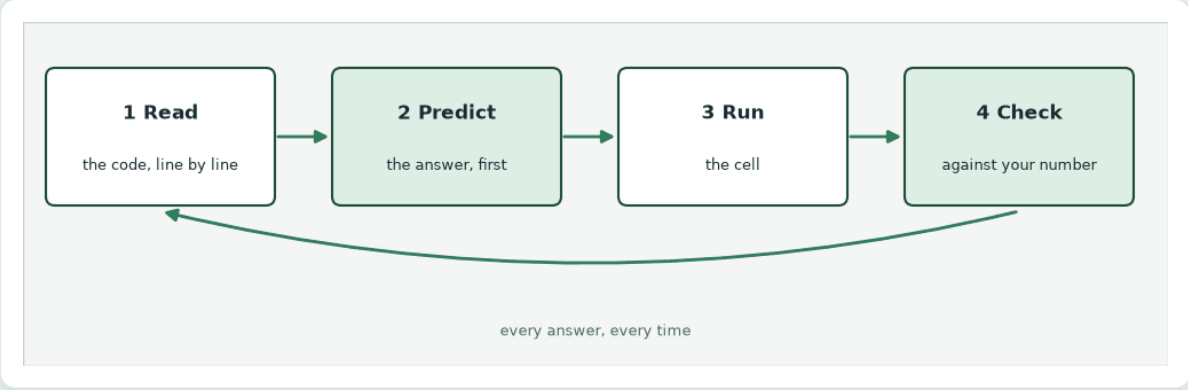
Excel side by side

Python	Excel	Result
`rating_text.strip()`	`=TRIM(A1)`	bbb
`rating_text.strip().upper()`	`=UPPER(TRIM(A1))`	BBB
`rating_clean.lower()`	`=LOWER(A1)`	bbb
`face_text.replace(',', '')`	`=SUBSTITUTE(A1,",","")`	1500000, still text
`int(face_text.replace(',', ''))`	`=VALUE(SUBSTITUTE(A1,",",""))`	1500000, a number

Working with an AI assistant

Never put into an AI tool or a cloud notebook	Fine to use
Client and counterparty names	The synthetic data in this course
Positions, holdings, and trades	Public data, such as Treasury yields
Anything from your employer: files, code, emails	Code you wrote on that data
Passwords, keys, and other credentials	

Code that runs without an error can still be wrong.



How to ask. Say what you have, what you want, and the units: "I have face amounts in dollars and prices per 100 face. Write code that totals the cost in dollars."

How to check. Read the code, predict the answer from a number you worked out yourself, run it, and compare.

The note, answered

The note asked	Answer
What 1,000,000 face of QVNE pays a year	52,500.00 dollars
What it cost	985,000.00 dollars
Total face	3,000,000
Price range	95.00 to 103.25
Clean the broker's text	' bbb ' became 'BBB'; '1,500,000' became 1500000

In Part 2 your code starts working for you: it decides which bonds are investment grade, runs one calculation over all four, and turns the coupon formula into a tool.